PromoFinder

A tool that reads a microdrama's screenplays and hands an editor a ranked list of 20-60 second promos, with exact cut points.

Live app: kuku-promo-finder.streamlit.app · Code: github.com/vatsal3512/kuku-promo-finder · Raw outputs: raw_outputs/ in the repo

Which problem, and why

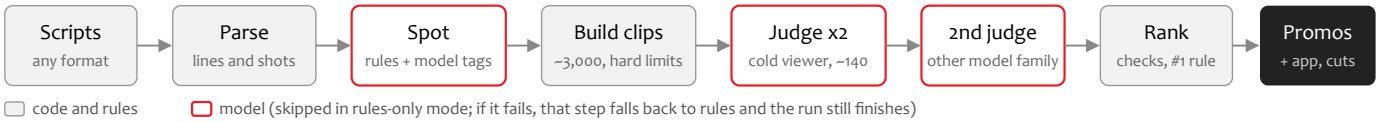
Problem 1. Today an editor watches every episode to find promo moments, about 100 minutes of footage for a 50-episode show, and Kuku releases new shows every week. The output is easy to check (a timestamp either works or it doesn't), the brief defines a good promo clearly, and promos are how paid viewers arrive, so better picks are worth real money. The editor stays in charge and still approves every cut, but starts from a ranked shortlist instead of raw footage.

What I assumed

- Script timestamps match the finished video, and CAMERA lines mark real shot changes, so cuts land on them.
- A promo's job is to bring in new viewers, so early-episode clips rank higher (viewers land near the start). Big late twists are kept but marked for retargeting existing viewers.
- "Hook in the first 3 seconds" means the hook starts within 3 s of the cut. "Ends on a cliffhanger" means the cut lands within 1.5 s after that line, never mid-line.
- "Makes sense on its own" is judged by a model that sees only the clip, never the show. Ad risks (violence, gore, coercion, minors) are flagged for a person, not silently removed.
- Free tiers only, and every model is one config value, so Kuku's own models can be plugged in.

How it works

The model never writes a timestamp. It points at lines in the script; code works out the cuts. I tested this: when the same model wrote timestamps itself, 23 of 100 promos were broken, and 22 of those stopped at the *start* of the cliffhanger line, cutting off the very moment the promo exists for.



- **Hard limits in code:** 20-60 s, hook within 3 s, cut right after the cliffhanger, never mid-line. A clip under 3/5 on "makes sense alone" or on cliffhanger is dropped.
- **Craft checks the judges miss** (they read the whole clip at once): no spoken line in the first 3 s, a one-word opening, a long silent tail, length over 50 s.
- **Ranking:** one promo per scene, pairwise model comparisons for the order. #1, the one to post first, must come from the first 20% of episodes, have a strong hook and cliffhanger, and carry the fewest ad flags.
- **Output per promo:** segments, hook, cliffhanger, why it works, why it has that rank, the episode it sends viewers to, ad flags; plus an edit list and ffmpeg cut scripts. The app shows each promo with a timeline and transcript, and the editor approves or rejects it.

> **Tech stack:** Python 3.12 · Pydantic (strict but forgiving reply schemas) · OpenAI-compatible client, any provider by one string · NVIDIA NIM free tier: Nemotron 3 Ultra (picks, judges), Mistral Large 4 (second judge), MiniMax M3.1 (held-out auditor) · Streamlit app on Streamlit Community Cloud · pytest (41 tests, no key needed) · GitHub.

How it scales, and what it costs

A 50-episode show takes about 3 minutes in **fast** mode (one judging pass) and 20-25 minutes in **full** mode (two passes, second judge, ordering), on NVIDIA's free tier. Full mode is about 140 model calls and 0.45M tokens; at paid rates of $1-3 per million tokens that is under $1.50 a show. Rules-only mode takes 5 seconds and costs nothing. Only tagging grows with episode count; judging is a fixed budget per show. Every answer is cached, so a crashed or changed run only pays for what's new. Shows run side by side; the limit is provider rate limits, so the tool rotates API keys, backs off when throttled, and stops at once on a bad key or empty quota. The editor reads about 12 minutes of shortlisted clips instead of 100 minutes of footage.

How I know it's good

Check	What I found
Are the cuts usable?	15 of 15 in every run. The model writing timestamps itself: 77 of 100.
Do outside judges agree?	Two models that never picked anything scored the lists blind (MiniMax M3.1 took no part in any run). Against the rules-only list, the final list had better hooks (4.53 vs 4.33) and cliffhangers (4.67 vs 4.47), and every promo passed the standalone check (87% for rules only; Mistral: 100% vs 93%).

Did self-grading fool me?	Yes, once. A mid-day "improvement" looked better to the picking model (0.76 to 0.85) but worse to Mistral (standalone pass 100% to 80%). That's why the second judge exists; with it the list went back to MiniMax's best score (0.80).
What does a person think?	Blind A/B, 8 pairs, tool cut vs rules-only cut: I preferred the tool's in 5 of 8. One person, so a hint, not proof. My three "no" votes became the craft checks.
Will it hold up on a new show?	Two made-up shows in other genres and formats (one-file SRT style; Hindi names): all promos valid. Broken files and fake models that fail like real ones (bad key, no credit, refusals, odd replies) all finish with output. A later robustness release scores 0.78 vs 0.80 blind, inside judge noise, so the submitted list stays official.

The top of the final list

#	Clip	Opens on / cuts on	Why it's here
1	Ep1 00:39-01:30, 52 s	“Because he is marrying me.” Ends as her father refuses to look at her.	The whole premise: jilted at the altar for her own sister. Nothing for ad review, lands viewers on episode 1. Post first.
2	Ep5 00:54-01:45, 52 s	Her mother's book thrown into the rain. Ends on the stranger's “Come.”	Works with the sound off; the scarred stranger steps in for her the first time.
3	Ep12 00:26-01:09, 43 s	“You planted it!” Ends on “Fail, or refuse, and your husband hangs.”	The sharpest choice in the show. Needs an ad-policy check (bare hands in thorns).
4	Ep3 01:24-02:15, 52 s	A stranger floors the guard. Ends as he kneels with a ring in the church.	The only early clip with the soldier from the title; ends on “will she say yes?”.
5	Ep11 00:16-01:15, 58 s	“Your husband stole my gold pocket watch!” Ends mid-search of their home.	Easy for a stranger to follow, but long at 58 s; test it against the shorter ones.

Where it picks badly

It reads; viewers watch. All three judges gave openings like a bare "No." or a silent hand on a bedcover (first word at 0:09) 4-5 out of 5, because they could see the tension coming later. Craft checks catch the measurable cases; whether a shot stops a thumb needs the video.

The judges are generous and noisy. Most scores are 4 or 5 and a clip can move a point between batches, so once my changes moved scores by less, I stopped tuning against them.

It repeats storylines. One-per-scene stops repeated scenes, not storylines: four of fifteen promos are the poison plot. My three tries at a fix didn't work.

Stage directions and names mislead. "She gasps, realizing her secret identity has been laid bare" tells the judge what a viewer never sees. The scripts also spell one estate six ways and reuse a speaker label for two people; the cast list fixes most of it, not all.

Where the human stays in the loop

The tool never posts anything: it shortlists and explains, the editor approves or rejects in the app. Ad risks are flagged, never silently dropped, and every decision is logged so weights can follow what editors approve, not what an AI judge likes.

Running it on a new show

In the app: (1) in the left panel, upload one script per episode, a .zip, or a whole series in one file; (2) pick **Fast** (AI, about 3 minutes for 50 episodes) or **Rules only** (seconds, no key); (3) click **Find promos** and watch the steps tick by; (4) the new run opens by itself; (5) click a promo on the left to see its cut, timeline, transcript and checks, then approve or reject it. A "How to use this app" box at the top explains each part. **From the command line:** `check --input new_show --model provider/model` to see how it parsed and test the model, then `run --profile fast`. A stronger model is one `--model` argument.

How I'd know it works once a team uses it

Ads: the tool's picks against the editor's on the same budget: 3-second view rate (hook), hold to the end (cliffhanger), cost per install, episode-2 retention. **Editors:** share approved unchanged, minutes per show. **Next:** check the first 3 seconds of video with a vision model, storyline labels, and weights learned from approvals and ad results.

AI tools I used, and where they got it wrong

I used Claude Code as a coding assistant, to write and debug code and run the experiments I asked for. On NVIDIA's free tier, Nemotron 3 Ultra picks and judges, Mistral Large 4 gives the second opinion, and MiniMax M3.1 audits at the end. The calls that shaped the tool were mine: staying on free NVIDIA models, holding the doc until the quality numbers improved, doing the blind A/B myself (my three "no" votes became the craft checks), and asking for a prototype that looks and reads like a real Kuku tool.

Where they got it wrong: the model that picked the clips also graded them, and said my changed version was better when it was worse. Only judges from other model families caught it, so I no longer let a picker grade its own work. Smaller misses: the first models suggested had been retired (HTTP 410), and the "post first" rule once put a clip with a weak cliffhanger at #1 because it checked ad safety before quality.